

Manual do módulo SDNVoIP

1. Introdução

Esta seção apresenta o módulo SDNVoIP, uma aplicação para controle de tráfego VoIP em SDN. A aplicação gerencia o tráfego VoIP, reduzindo a utilização de CPU em servidores e a largura de banda utilizada no segmento do servidor VoIP em redes SDN. Para reduzir o uso de CPU nos servidores, SDNVoIP reconfigura as chamadas que utilizem o mesmo *Codec* para realizarem uma comunicação fim-a-fim, sem passar pelo servidor. Ou seja, o cenário da Figura 1(a) é transformado no cenário (b), sem a necessidade reconfigurar os servidores VoIP.

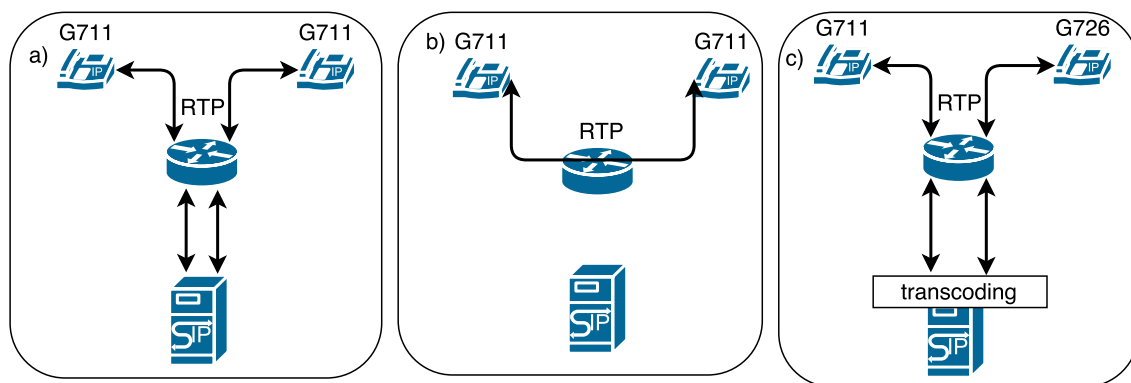


Figura 1. Exemplos de tráfego VoIP com *Codec* único (a e b) e com *Codecs* distintos (c).

A Figura 1 apresenta três maneiras pelas quais pode ocorrer o tráfego de dados em chamadas VoIP: (a) Os dados de uma chamada com o mesmo *Codec* de áudio (G711, por exemplo) são trafegados de um cliente para o outro através do servidor VoIP. Nesse caso o servidor está no caminho do áudio e dois canais são criados, um para cada cliente. O servidor VoIP lê o áudio de um canal e escreve no outro, aumentando o uso de memória e de CPU durante a leitura e escrita dos canais. Essa configuração é comumente utilizada para monitorar chamadas, coletando informações relacionadas com a qualidade do atendimento prestado por centrais de vendas. (b) O tráfego de sinalização de clientes com o mesmo *Codec* de áudio e os dados de voz são trafegados diretamente entre os clientes. Nesse caso, o servidor VoIP não está no caminho do áudio e não é possível gravar a chamada, acrescentar música de fundo, transferir ou colocar a chamada em espera. Entretanto, essa abordagem reduz o uso de CPU, memória alocada e largura de banda utilizada pelo servidor. (c) Clientes com *Codecs* diferentes e os dados são trafegados sempre pelo servidor VoIP. Semelhante ao cenário (a), o servidor está no caminho do áudio e dois canais são criados, um para cada cliente. Entretanto, o servidor VoIP efetua a tradução dos pacotes de voz de acordo com os *Codecs* utilizados para que os clientes possam receber e ouvir o áudio [Goode 2002], aumentando o consumo dos recursos do servidor.

2. Requisitos de software

Para instalar e utilizar o módulo SDNVoIP é necessário os seguintes *softwares*:

- Java 1.7 ou posterior
- Eclipse 4.5 ou posterior
- Git

O código fonte pode ser encontrado em:

<https://github.com/paulorvj/sdnvoip>

O projeto foi desenvolvido como um módulo do controlador Floodlight [Project Floodlight 2016] e o código disponível no *link* acima possui um projeto completo e configurado para ser importado na IDE Eclipse, não sendo necessário fazer a instalação isolada do controlador Floodlight. Informações sobre como instalar e construir o controlador Floodlight podem ser encontradas em:

<https://floodlight.atlassian.net/wiki/spaces/floodlightcontroller/pages/1343544/Installation+Guide>

Informações sobre como desenvolver um módulo para o Floodlight podem ser encontradas em:

<https://floodlight.atlassian.net/wiki/spaces/floodlightcontroller/pages/1343513/How+to+Write+a+Module>

2.1. Importando o projeto

Primeiro é necessário fazer um clone do repositório Git, para isso utilize o comando (Figura 2):

```
# git clone https://github.com/paulorvj/sdnvoip.git
```

```
wy63xzy-2:sdnvoip paulorvj$ git clone https://github.com/paulorvj/sdnvoip.git
Cloning into 'sdnvoip'...
remote: Counting objects: 2917, done.
remote: Compressing objects: 100% (896/896), done.
remote: Total 2917 (delta 1971), reused 2917 (delta 1971), pack-reused 0
Receiving objects: 100% (2917/2917), 53.06 MiB | 2.38 MiB/s, done.
Resolving deltas: 100% (1971/1971), done.
wy63xzy-2:sdnvoip paulorvj$
```

Figura 2. Git clone.

Após fazer o clone, abra o Eclipse e vá em “File - Import - Existing projects into workspace” (Figura 3):

Selecione o diretório onde foi realizado o clone do repositório, marque o nome do projeto e clique em “Finish” (Figura 4):

Após a importação o projeto estará pronto para execução no Eclipse. Para executar o controlador clique em “Run - Run configurations”, clique em “Java Application” e depois no botão “New launch configuration” e em “Main class” digite (Figura 5):

```
net.floodlightcontroller.core.Main
```

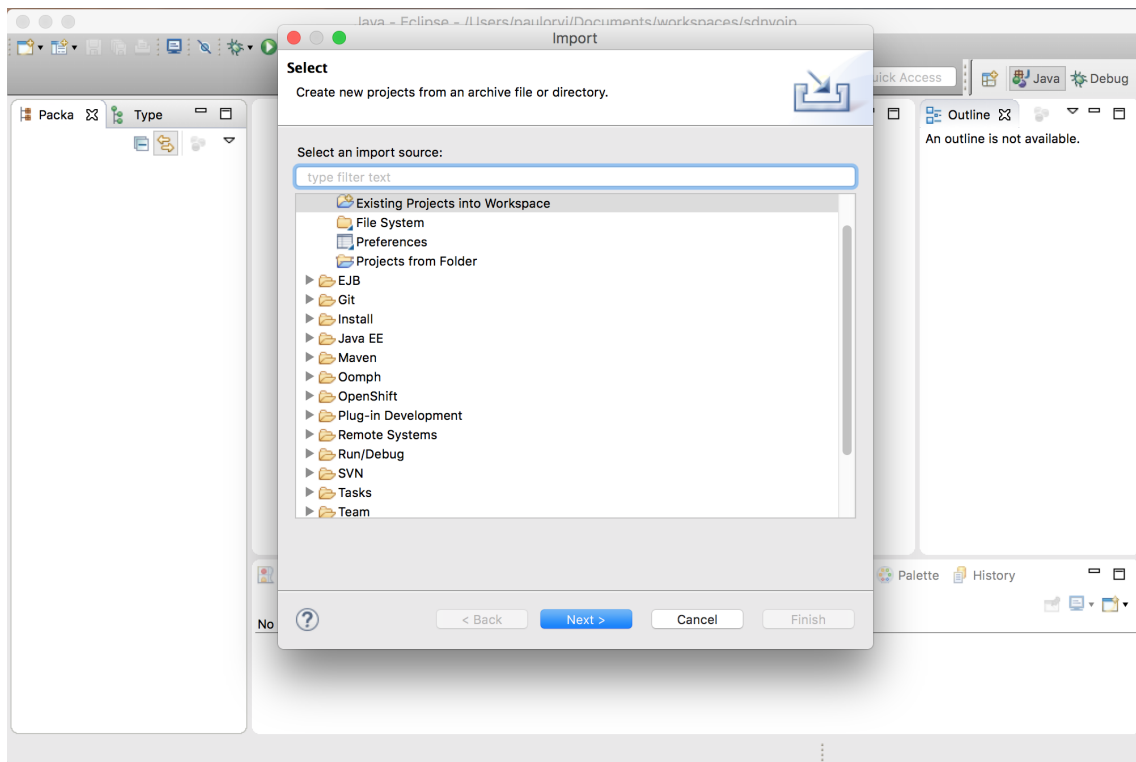


Figura 3. Importando o projeto no eclipse.

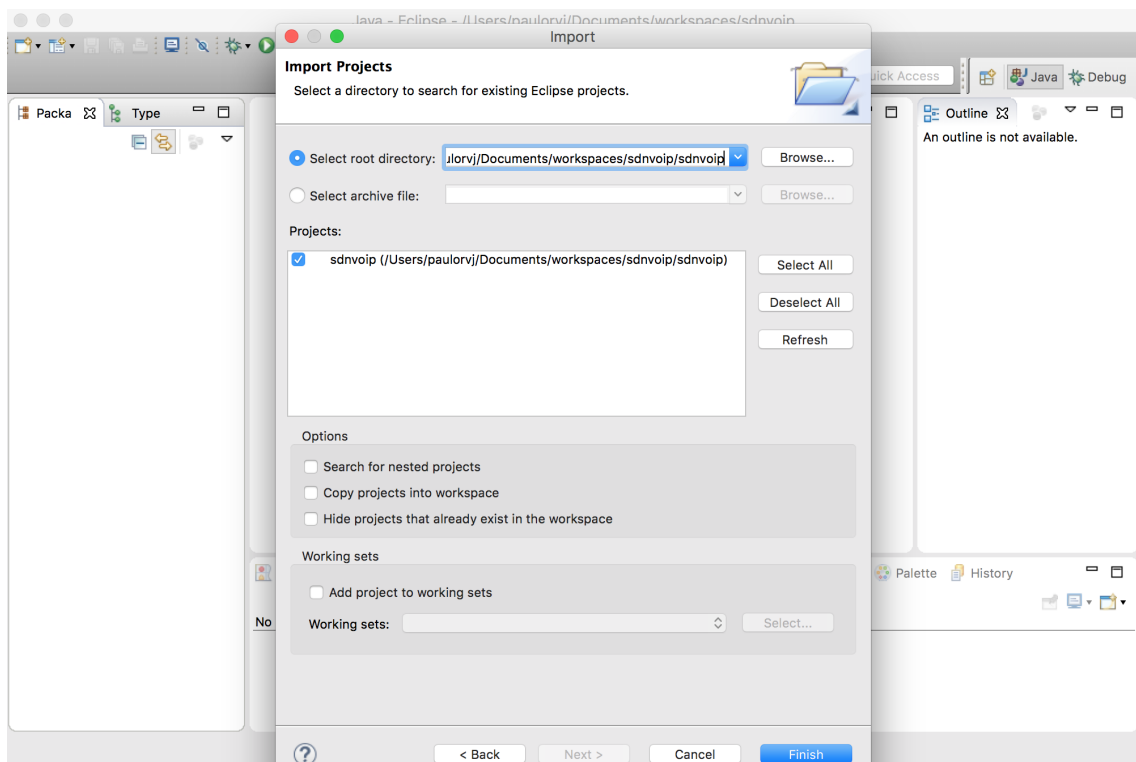


Figura 4. Importando o projeto no eclipse.

Clique em “Run” e caso não aconteça nenhum erro o controlador será executado

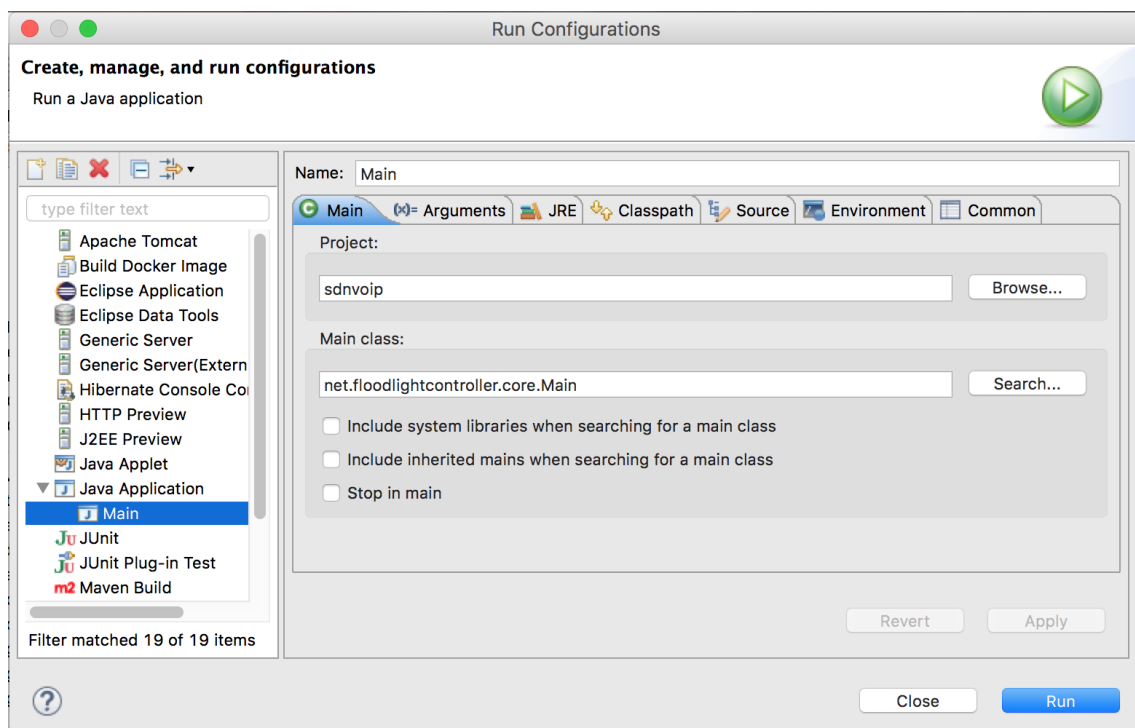


Figura 5. Run configuration.

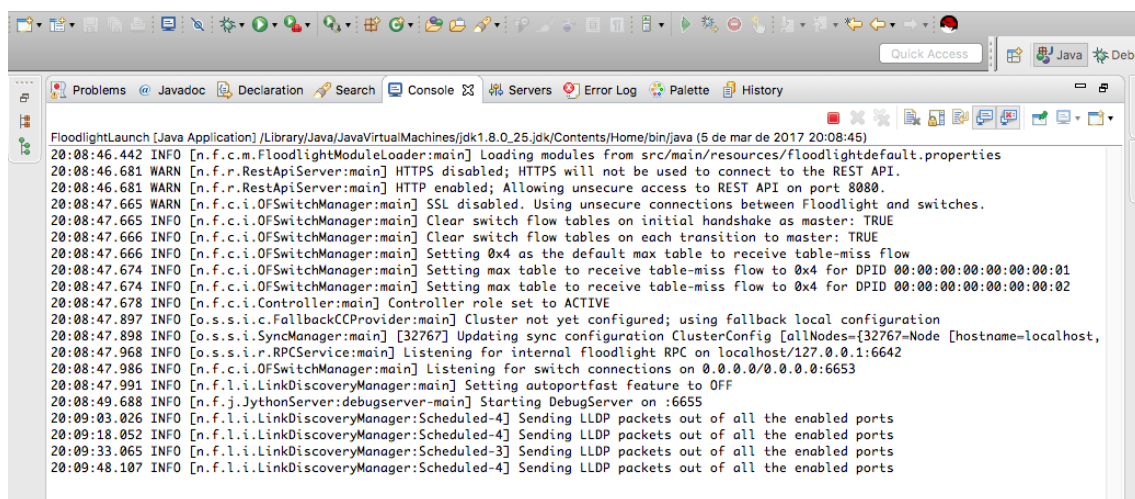


Figura 6. Execução do controlador

(Figura 6):

A partir desse momento o controlador está pronto para detectar chamadas VoIP que utilizem o mesmo Codec, não é necessário nenhuma configuração adicional.

2.2. Arquivos importantes

O código fonte do módulo encontra-se no seguinte caminho:

/src/main/java/net/floodlightcontroller/voip/Voip.java

Para que o módulo funcione corretamente é necessário que os IPs dos servidores

VoIP sejam colocados em um arquivo texto, o arquivo está localizado no seguinte caminho e deve conter um IP por linha:

/sdnvoip/src/main/resources/voipservers.txt

O módulo SDNVoIP mantém informações dos clientes que estão executando chamadas, permitindo a instalação dos fluxos nos *switches* OpenFlow no caminho entre eles.

Quatro fluxos por chamada são instalados nos *switches* SDN. Dois fluxos para o caminho no sentido origem-destino e destino-origem dos pacotes RTP; e dois fluxos para o caminho no sentido origem-destino e destino-origem dos pacotes RTCP. Para cada fluxo VoIP, sete ações [Open Networking Foundation 2012] são aplicadas nos pacotes RTP e RTCP que trafegam no *switch* correspondente:

- (i) Alterar o endereço MAC de origem para o endereço MAC do servidor VoIP;
- (ii) Alterar o endereço MAC de destino para o endereço MAC do cliente destino;
- (iii) Alterar o endereço IP de origem para o endereço IP do servidor VoIP;
- (iv) Alterar o endereço IP de destino para o endereço IP do cliente destino;
- (v) Alterar a porta RTP/RTCP de origem para a porta RTP/RTCP do servidor VoIP;
- (vi) Alterar a porta RTP/RTCP de destino para a porta RTP/RTCP do cliente destino;
- (vii) Saída do pacote em uma porta física do *switch*.

As entradas nas tabelas de fluxos devem ser instaladas nos *switches* que compõem o caminho da chamada com um tempo de expiração. Quando o término de uma chamada é sinalizado pelos clientes, a entrada é removida. Em caso de interrupção abrupta da chamada, as entradas relacionadas são automaticamente removidas dos *switches* após o tempo de expiração.

2.3. Topologia

Para demonstrar a solução em um cenário com SDN a seguinte topologia foi utilizada (Figura 7): um servidor VoIP executando Asterisk 13.01, um *switch* TP-Link WR1043 com OpenWRT e OVS habilitados, um controlador, um notebook executando *softphone* Linphone representando o cliente origem (CO) e por fim, um PC executando *softphone* Linphone representando o cliente destino (CD).

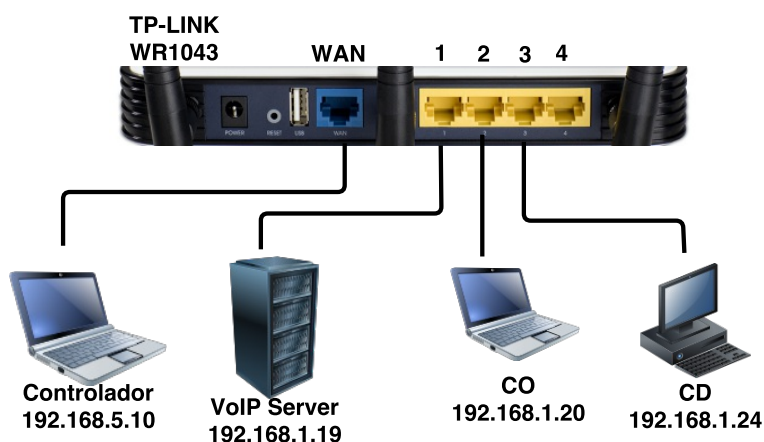


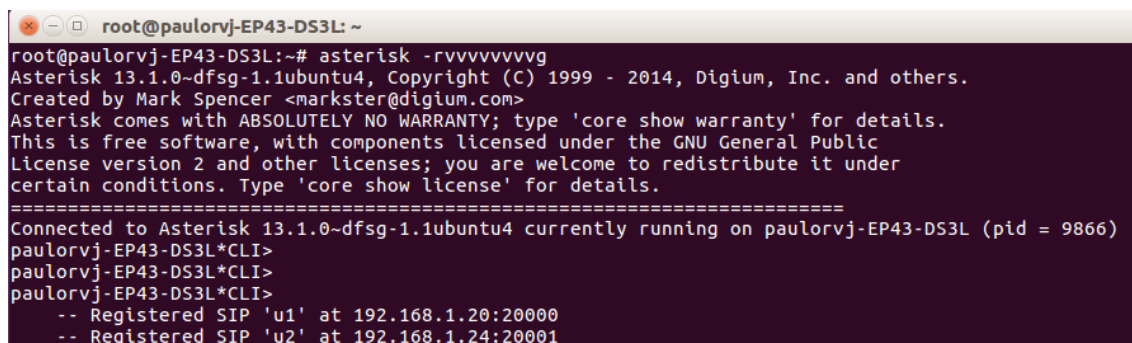
Figura 7. Cenário experimental com SDN.

A chamada feita entre CO e CD é realizada utilizando o mesmo *Codec* (G711) e todo o áudio por padrão é trafegado passando pelo servidor VoIP.

2.4. Chamadas VoIP sem SDN

A primeira demonstração é com uma chamada entre CO e CD sem a utilização de SDN, dessa maneira todo o áudio da chamada passa pelo servidor VoIP.

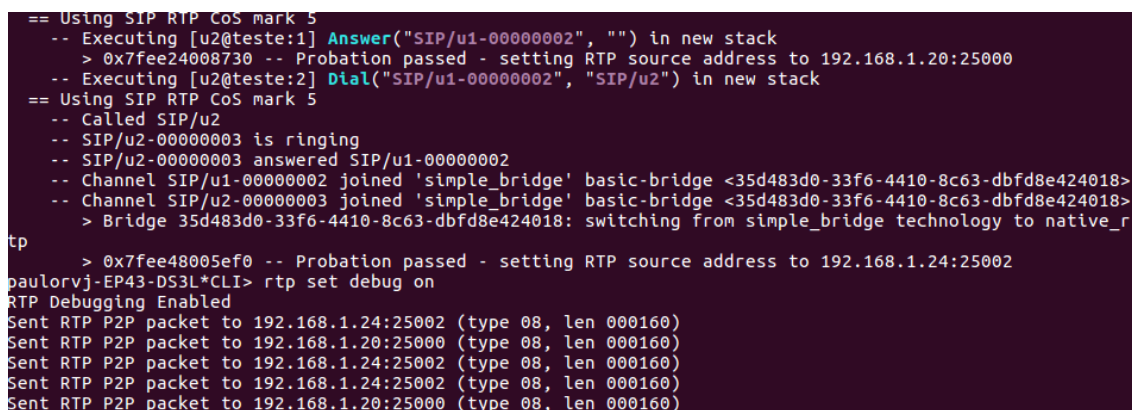
O servidor VoIP deve estar em execução e os dois clientes são iniciados, podemos observar que os dois clientes (u1 e u2) iniciam uma sessão no servidor VoIP (Figura 8):



```
root@paulorvj-EP43-DS3L: ~
root@paulorvj-EP43-DS3L:~# asterisk -rvvvvvvvvg
Asterisk 13.1.0~dfsg-1.1ubuntu4, Copyright (C) 1999 - 2014, Digium, Inc. and others.
Created by Mark Spencer <markster@digium.com>
Asterisk comes with ABSOLUTELY NO WARRANTY; type 'core show warranty' for details.
This is free software, with components licensed under the GNU General Public
License version 2 and other licenses; you are welcome to redistribute it under
certain conditions. Type 'core show license' for details.
=====
Connected to Asterisk 13.1.0~dfsg-1.1ubuntu4 currently running on paulorvj-EP43-DS3L (pid = 9866)
paulorvj-EP43-DS3L*CLI>
paulorvj-EP43-DS3L*CLI>
paulorvj-EP43-DS3L*CLI>
-- Registered SIP 'u1' at 192.168.1.20:20000
-- Registered SIP 'u2' at 192.168.1.24:20001
```

Figura 8. Registro dos clientes no servidor VoIP

Em seguida o cliente u1 inicia uma chamada para o cliente u2, observa-se na Figura 9 o início da chamada e o tráfego RTP passando pelo servidor VoIP:



```
== Using SIP RTP CoS mark 5
-- Executing [u2@teste:1] Answer("SIP/u1-00000002", "") in new stack
> 0x7fee24008730 -- Probation passed - setting RTP source address to 192.168.1.20:25000
-- Executing [u2@teste:2] Dial("SIP/u1-00000002", "SIP/u2") in new stack
== Using SIP RTP CoS mark 5
-- Called SIP/u2
-- SIP/u2-00000003 is ringing
-- SIP/u2-00000003 answered SIP/u1-00000002
-- Channel SIP/u1-00000002 joined 'simple_bridge' basic-bridge <35d483d0-33f6-4410-8c63-dbfd8e424018>
-- Channel SIP/u2-00000003 joined 'simple_bridge' basic-bridge <35d483d0-33f6-4410-8c63-dbfd8e424018>
> Bridge 35d483d0-33f6-4410-8c63-dbfd8e424018: switching from simple_bridge technology to native_r
tp
> 0x7fee48005ef0 -- Probation passed - setting RTP source address to 192.168.1.24:25002
paulorvj-EP43-DS3L*CLI> rtp set debug on
RTP Debugging Enabled
Sent RTP P2P packet to 192.168.1.24:25002 (type 08, len 000160)
Sent RTP P2P packet to 192.168.1.20:25000 (type 08, len 000160)
Sent RTP P2P packet to 192.168.1.24:25002 (type 08, len 000160)
Sent RTP P2P packet to 192.168.1.24:25002 (type 08, len 000160)
Sent RTP P2P packet to 192.168.1.20:25000 (type 08, len 000160)
```

Figura 9. Chamada VoIP e tráfego RTP

2.5. Chamadas VoIP com SDN

Para a demonstração com SDN foi executado o controlador Floodlight e feita a reconfiguração do *switch* TP-Link WR1043 com OpenWRT e OVS habilitados e comunicando-se com o controlador. A Figura 10 apresenta no retângulo vermelho a conexão do *switch* com o controlador e no retângulo verde a descoberta dos dispositivos na rede:

Assim como no cenário sem SDN, todo o áudio entre os clientes é trafegado pelo servidor VoIP. Entretanto, após o controlador obter todas as informações necessárias, quatro fluxos para cada chamada são instalados no *switch*, fazendo com que o áudio das chamadas não trafegue pelo servidor VoIP, fluindo diretamente entre os clientes. O módulo detecta a chamada feita entre CO e CD e instala os fluxos necessários no *switch* fazendo

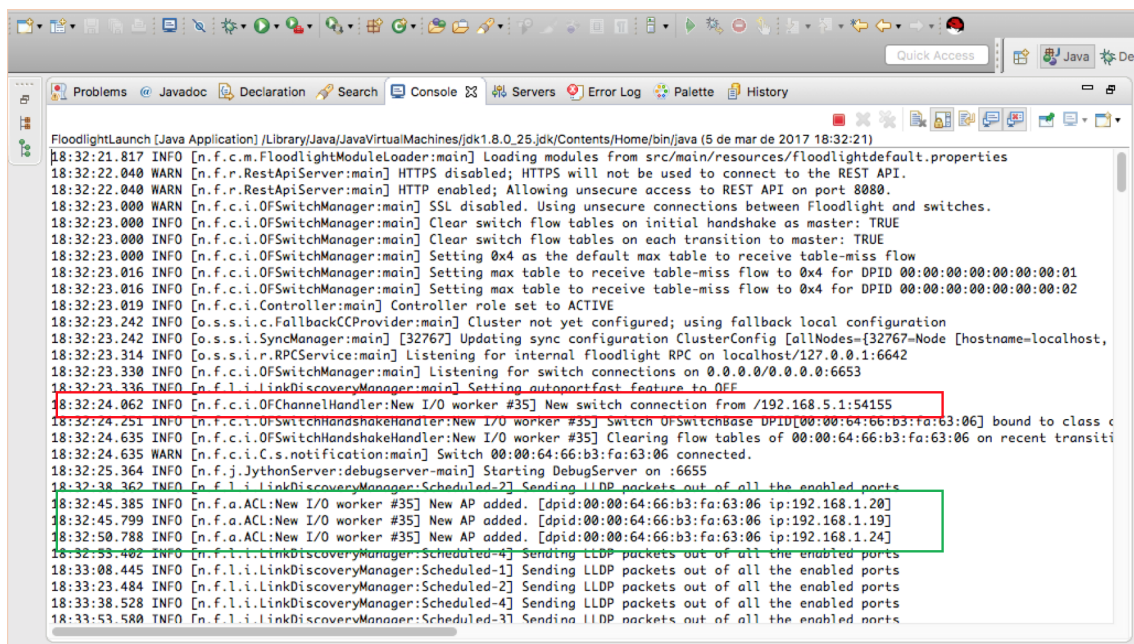


Figura 10. Execução do controlador

com que o tráfego de voz ocorra diretamente entre os clientes. A Figura 11 apresenta a chamada entre os clientes CO e CD, podemos observar que alguns pacotes RTP passam pelo servidor até o momento em que os fluxos são instalados no *switch* e a partir desse momentos os pacotes são trafegados diretamente entre os clientes.

```
paulorvj-EP43-DS3L*CLI>
== Using SIP RTP CoS mark 5
-- Executing [u2@teste:1] Answer("SIP/u1-00000004", "") in new stack
-- Executing [u2@teste:2] Dial("SIP/u1-00000004", "SIP/u2") in new stack
== Using SIP RTP CoS mark 5
-- Called SIP/u2
-- SIP/u2-00000005 is ringing
Sent RTP packet to 192.168.1.20:25000 (type 08, seq 006537, ts 000160, len 000160)
Sent RTP packet to 192.168.1.20:25000 (type 08, seq 006538, ts 000320, len 000160)
Sent RTP packet to 192.168.1.20:25000 (type 08, seq 006539, ts 000480, len 000160)
Sent RTP packet to 192.168.1.20:25000 (type 08, seq 006540, ts 000640, len 000160)
Sent RTP packet to 192.168.1.20:25000 (type 08, seq 006541, ts 000800, len 000160)
-- SIP/u2-00000005 answered SIP/u1-00000004
-- Channel SIP/u1-00000004 joined 'simple_bridge' basic-bridge <5a62b331-7535-4d98-8c1a-c6b5e5051aed>
-- Channel SIP/u2-00000005 joined 'simple_bridge' basic-bridge <5a62b331-7535-4d98-8c1a-c6b5e5051aed>
> Bridge 5a62b331-7535-4d98-8c1a-c6b5e5051aed: switching from simple_bridge technology to native_r
tp
-- Channel SIP/u1-00000004 left 'native_rtp' basic-bridge <5a62b331-7535-4d98-8c1a-c6b5e5051aed>
-- Channel SIP/u2-00000005 left 'native_rtp' basic-bridge <5a62b331-7535-4d98-8c1a-c6b5e5051aed>
== Spawn extension (teste, u2, 2) exited non-zero on 'SIP/u1-00000004'
-- Unregistered SIP 'u1'
-- Unregistered SIP 'u2'
paulorvj-EP43-DS3L*CLI>
```

Figura 11. Chamada com o módulo SDNVoIP ativo

A Figura 12 apresenta os pacotes RTP recebidos e enviados por ambos os clientes CO e CD após a instalação dos fluxos no *switch*:

Um video demonstrando o funcionamento do módulo pode ser encontrado em:

https://www.youtube.com/watch?v=f9tir_EPIIU

No.	Source	Destination	Src Port	Dst Port	Protocol
1571	192.168.1.20	192.168.1.19	25000	17604	RTP
1572	192.168.1.19	192.168.1.20	17604	25000	RTP
1573	192.168.1.20	192.168.1.19	25000	17604	RTP
1574	192.168.1.19	192.168.1.20	17604	25000	RTP
1575	192.168.1.19	192.168.1.20	17604	25000	RTP
1576	192.168.1.20	192.168.1.19	25000	17604	RTP
1577	192.168.1.19	192.168.1.20	17604	25000	RTP
1578	192.168.1.20	192.168.1.19	25000	17604	RTP
1579	192.168.1.20	192.168.1.19	25000	17604	RTP
1580	192.168.1.19	192.168.1.20	17604	25000	RTP
1581	192.168.1.20	192.168.1.19	25000	17604	RTP
1582	192.168.1.19	192.168.1.20	17604	25000	RTP
1583	192.168.1.20	192.168.1.19	25000	17604	RTP
1584	192.168.1.19	192.168.1.20	17604	25000	RTP
1585	192.168.1.20	192.168.1.19	25000	17604	RTP
1586	192.168.1.19	192.168.1.20	17604	25000	RTP

No.	Source	src port	Destination	dst port	Protc
1544	192.168.1.19	18824	192.168.1.24	25002	RTP
1545	192.168.1.19	18824	192.168.1.24	25002	RTP
1546	192.168.1.24	25002	192.168.1.19	18824	RTP
1547	192.168.1.19	18824	192.168.1.24	25002	RTP
1548	192.168.1.24	25002	192.168.1.19	18824	RTP
1549	192.168.1.24	25002	192.168.1.19	18824	RTP
1550	192.168.1.19	18824	192.168.1.24	25002	RTP
1551	192.168.1.24	25002	192.168.1.19	18824	RTP
1552	192.168.1.19	18824	192.168.1.24	25002	RTP
1553	192.168.1.19	18824	192.168.1.24	25002	RTP
1554	192.168.1.24	25002	192.168.1.19	18824	RTP
1555	192.168.1.19	18824	192.168.1.24	25002	RTP
1556	192.168.1.24	25002	192.168.1.19	18824	RTP
1557	192.168.1.19	18824	192.168.1.24	25002	RTP
1558	192.168.1.24	25002	192.168.1.19	18824	RTP
1559	192.168.1.24	25002	192.168.1.19	18824	RTP

Figura 12. Pacotes RTP em CO e CD

Referências

- Chen, W.-E., Huang, Y.-L., and Chao, H.-C. (2008). Nat traversing solutions for sip applications. *EURASIP Journal on Wireless Communications and Networking*, 2008(1):1–9.
- Goode, B. (2002). Voice over internet protocol (voip). *Proceedings of the IEEE*, 90(9):1495–1517.
- Handley, M. (2006). Why the internet only just works. *BT Technology Journal*, 24(3):119–129.
- Jain, R. and Paul, S. (2013). Network virtualization and software defined networking for cloud computing: A survey. *IEEE Communications Magazine*, 51(11):24–31.
- Karapantazis, S. and Pavlidou, F. N. (2009). VoIP: A comprehensive survey on a promising technology. *Computer Networks*, 53(12):2050–2090.
- Khurul, I., Islam, M. K., and Hasan, K. A. (2007). An efficient approach for nat traversal problem on security of voice over internet protocol. In *10th International Conference on Computer and information technology*, 2007, pages 1–5. IEEE.
- Khelifi, H., Gregoire, J., and Phillips, J. (2006). Voip and nat/firewalls: issues, traversal techniques, and a real-world solution. *IEEE Communications Magazine*, 44(7):93.
- Kreutz, D., Ramos, F. M. V., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., and Uhlig, S. (2014). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76.
- Lin, Y.-D., Tseng, C.-C., Ho, C.-Y., and Wu, Y.-H. (2010). How nat-compatible are voip applications? *IEEE Communications Magazine*, 48(12):58–65.
- McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., and Turner, J. (2008). Openflow: enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2):69–74.
- Open Networking Foundation (2012). OpenFlow Switch Specification. <http://www.cs.yale.edu/homes/yu-minlan/teach/csci599-fall12/papers/openflow-spec-v1.3.0.pdf>.

- Park, C., Jeong, K., Kim, S., and Lee, Y. (2008). Nat issues in the remote management of home network devices. *IEEE network*, 22(5):48–55.
- Project Floodlight (2016). Project floodlight: Open source software for building software-defined networks. <http://www.projectfloodlight.org/floodlight>.
- Yeryomin, Y., Evers, F., and Seitz, J. (2008). Solving the firewall and nat traversal issues for sip-based voip. In *Telecommunications (ICT), International Conference on*, pages 1–6. IEEE.